Sentiment Analysis with an RNN in Keras

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. Integer encoding . . . 3
   - 3.1 The three steps . . . 3
   - 3.2 Tokenizing in Keras . . . 4
4. The IMDB dataset . . . 5
   - 4.1 Loading the reviews . . . 5
   - 4.2 Cutting every review to 50 words . . . 6
5. Approach 1: integer encoding into a SimpleRNN . . . 7
   - 5.1 The model . . . 7
   - 5.2 Counting the parameters . . . 7
   - 5.3 `return_sequences` . . . 7
   - 5.4 Training . . . 8
6. Word embeddings . . . 9
   - 6.1 Sparse and dense representations . . . 9
   - 6.2 Learning the embedding with the model . . . 9
   - 6.3 What the layer computes . . . 10
7. Approach 2: an embedding into the same SimpleRNN . . . 11
   - 7.1 The model . . . 11
   - 7.2 Results . . . 12
   - 7.3 What the embedding learned . . . 12
   - 7.4 Learned or pre-trained embeddings . . . 15
8. Summary . . . 16
9. Sources . . . 16
10. Key terms . . . 17

Where this fits: Pipeline map step 5 (Engineer features). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	**5 Features**	6 Reduce
7 Split	8 Model	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Recurrent neural network (RNN).
- **Leads to:** Next-word prediction with an LSTM; Meaning as direction in embedding space; Contextual embeddings.
- **Compare with:** One-hot encoding.

1. Overview

Key point: An RNN only reads numbers, so text must become numbers first. Two ways: integer encoding (each word becomes its index in the **vocabulary** (G-2092)) and a learned embedding (each word becomes a short, dense vector, a list of numbers, that the network learns). On IMDB movie reviews, the same SimpleRNN reaches about 0.80 test accuracy with an embedding, against 0.50 with raw integers.

Sentiment analysis (G-1769) is the task of predicting whether a text is positive or negative. The data is a set of texts, each with a label: 1 for positive, 0 for negative. This Note builds a sentiment analysis model with the RNN of the RNN forward propagation, in Keras, on real movie reviews.

The aim is the workflow, not the best accuracy:

- how text becomes numbers (sections 3 and 4);
- how the numbers enter a `SimpleRNN` (section 5);
- what changes when a learned embedding is added (sections 6 and 7).

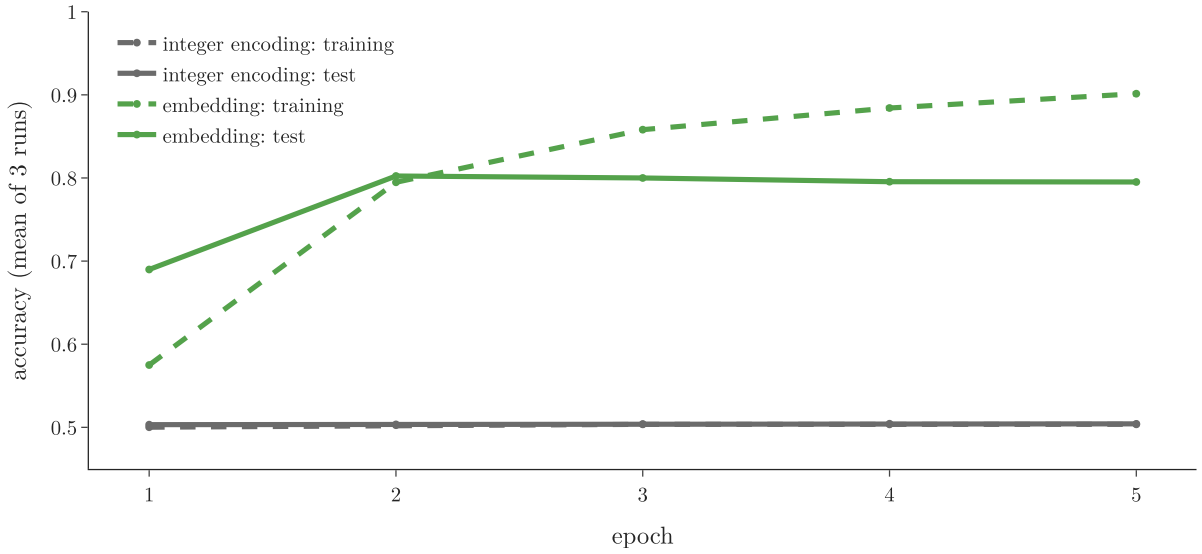


Figure 1: IMDB reviews cut to their last 50 words. Dashed: training accuracy; solid: test accuracy; mean of 3 runs. Raw integer inputs stay near a coin toss; a learned 2-number embedding lifts the same SimpleRNN to about 0.80 on the test reviews

Figure 1 is the result of the whole Note. The horizontal axis is the **epoch** (G-696) (one pass over the training reviews) and the vertical axis is accuracy, the share of reviews classified correctly. The grey lines are the model fed raw integers and the green lines the model with an embedding; watch how far apart the two solid lines end up.

2. Prerequisites

- The RNN forward propagation: input shape (time steps, input features), the recurrent layer, counting its parameters.
- The why RNNs: **padding** (G-1436) sequences to a common length, and its cost.
- The MNIST ANN: `compile`, `fit` and validation data in Keras.
- The **one-hot encoding** (G-1379): a category as a vector with a single 1.

3. Integer encoding

> **Key point:** Build the **vocabulary** (G-2092), give each word an integer, replace every word by its integer, then pad the sequences to one length.

3.1 The three steps

> **Key point:** Vocabulary, replace, pad.

Each review is one **observation** (G-1374; one record of the data), and its label is the **target** (G-1949; the output we predict). Before an RNN can read the reviews, every word must

become a number. **Integer encoding** (G-956) does it in three steps.

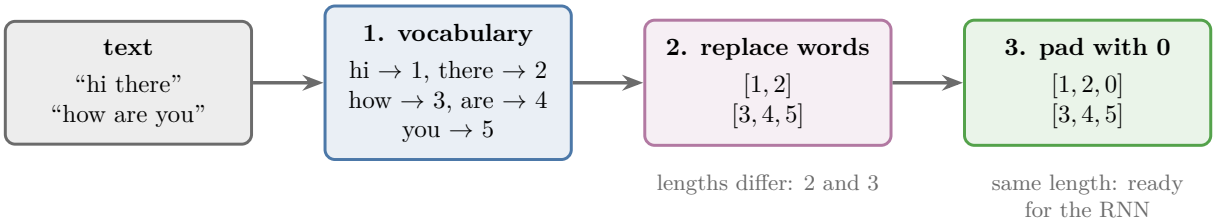


Figure 2: Integer encoding of two sentences: build the vocabulary and number its words, replace every word by its number, then pad the shorter sequence with 0

1. **Vocabulary.** List the unique words of the whole document, the **vocabulary** (G-2092), and give each one an integer: hi is 1, there is 2, how is 3, are is 4, you is 5.
2. **Replace.** Write every sentence as the integers of its words: "hi there" becomes $[1, 2]$ and "how are you" becomes $[3, 4, 5]$.
3. **Pad.** The sequences have different lengths, so we use **padding** (G-1436): add zeros at the start or the end of the shorter ones until all have the same length: $[1, 2, 0]$ and $[3, 4, 5]$. The integer 0 is kept for padding, so no word gets it.

Figure 2 shows the three steps.

3.2 Tokenizing in Keras

> **Key point:** Keras' `TextVectorization` layer lower-cases the text, strips punctuation, splits it into words, builds the **vocabulary** (G-2092) and replaces each word by its index, in one step.

Splitting a text into words is **tokenization** (G-1983). Doing it by hand means lower-casing every word, removing punctuation and numbering the words; Keras does all of it.

Take a document of 10 short cheering slogans, such as "go india", "india india" and "jeetega bhai jeetega india jeetega". `adapt` reads the document and builds the vocabulary, most frequent word first. The Notebook prints 19 entries:

Index	Entry
0	`""` (padding)
1	`[UNK]` (unknown word)
2	india
3	jeetega
...	15 more words

"india" appears 4 times, more than any other word, so it gets the first free index, 2.

Out-of-vocabulary words. A word seen only at prediction time, never in training, has no index. The layer maps every such word to a special **out-of-vocabulary (OOV) token** (G-1416), `[UNK]`, with index 1. In the Notebook, "go pakistan" becomes $[16, 1]$: "pakistan" is not in the vocabulary.

> **Python: Integer encoding** (G-956) with `TextVectorization`; it also pads the sequences with 0 at the end.
>
> ```
> import keras
> docs = ["go india", "india india", "hip hip hurray", ...]
> vectorizer = keras.layers.TextVectorization()
> vectorizer.adapt(docs)                # build the vocabulary
> print(vectorizer.get_vocabulary())    # ['', '[UNK]', 'india', ...]
> sequences = vectorizer(docs).numpy() # one padded row per sentence
> ```

For lists of integers of different lengths, `keras.utils.pad_sequences(sequences, padding="post")` pads at the end, and `padding="pre"` (the default) at the start (Keras documentation).

> **Extra:** Older Keras code uses `keras.preprocessing.text.Tokenizer` with `fit_on_texts`, `word_index`, `word_counts` and `texts_to_sequences`, and an `oov_token` argument. Keras 3 keeps that class only as a legacy API; `TextVectorization` is its replacement, and it reserves index 0 for **padding** (G-1436) and index 1 for `[UNK]` (Keras documentation, `TextVectorization`).

4. The IMDB dataset

> **Key point:** 50,000 movie reviews, half for training and half for testing, already integer encoded. We pad or cut every review to 50 words.

4.1 Loading the reviews

> **Key point:** `keras.datasets.imdb.load_data()` returns the reviews as lists of integers, one integer per word.

The **IMDB dataset** (G-923) holds 50,000 movie reviews from the Internet Movie Database, labelled positive or negative: 25,000 for training and 25,000 for testing (Keras documentation, IMDB dataset). Keras ships it already tokenized and integer encoded, so the steps of section 3 are done. The first training review starts

$$[1, 14, 22, 16, 43, 530, 973, 1622, 1385, 65, \dots]$$

where 1 marks the start of a review and every other integer is a word. Decoded with the dataset's word index, it reads "this film was just brilliant casting location scenery story direction everyone's really suited the part they played and you . . . ".

Extra: In `load_data`, a word's integer is its frequency rank plus 3: 0 is **padding** (G-1436), 1 is the start marker and 2 is the out-of-vocabulary word. So the most frequent word, "the", is 4 (Keras source, `imdb.load_data`: `start_char=1`, `oov_char=2`, `index_from=3`).

4.2 Cutting every review to 50 words

Key point: Reviews have different lengths (the first three have 218, 189 and 141 words). `pad_sequences(maxlen=50)` pads the short ones and cuts the long ones to 50 words, so training is fast.

Training on full reviews is slow, so we keep 50 words per review: `keras.utils.pad_sequences(X, maxlen=50)`. Shorter reviews get zeros in front; longer reviews are cut. Zeros in front keep the review's own words at the end, so the last hidden state, the one the output node reads, comes right after the last real word; with zeros at the end the RNN would first run extra steps on **padding** (G-1436), which change the hidden state (see padded steps still run). The training data then has shape $(25000, 50)$. The cut throws information away: the median review has 178 words. When training time allows, use full reviews.

Which 50 words? `pad_sequences` cuts from the start by default (`truncating="pre"`), so a long review keeps its **last** 50 words, not its first 50 (Keras documentation, `pad_sequences`). Passing `truncating="post"` keeps the first 50. Figure 3 shows both cases on real training reviews.

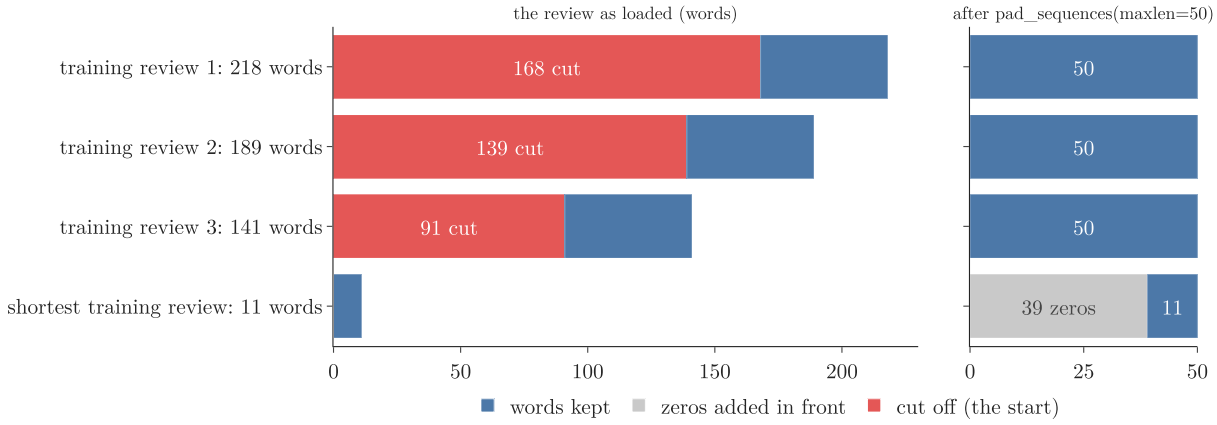


Figure 3: `pad_sequences(maxlen=50)` on four IMDB training reviews. The three long ones lose their start (red) and keep their last 50 words (blue); the shortest review, 11 words, gets 39 zeros in front

5. Approach 1: integer encoding into a SimpleRNN

> **Key point:** Feed each review as 50 time steps of 1 number each, into a SimpleRNN of 32 nodes and a sigmoid output node: 1,088 + 33 parameters.

5.1 The model

> **Key point:** Input shape $(50, 1)$: 50 time steps, 1 input feature, the word's integer.

> **Python:** The model for raw integer inputs.

```
model = keras.Sequential([
    keras.Input(shape=(50, 1)),     # (time steps, input features)
    keras.layers.SimpleRNN(32, return_sequences=False),
    keras.layers.Dense(1, activation="sigmoid")])
```

Each review is 50 integers. At time step 1 the first integer enters the recurrent layer, at time step 2 the second, and so on, as in the RNN forward propagation. After the 50th, the last hidden state goes to the sigmoid node, which gives $\hat{y}$. The **hidden state** is the vector of numbers the recurrent layer keeps as its memory and passes from each word to the next (how it carries the sequence forward); $\hat{y}$ is the predicted probability that the review is positive.

5.2 Counting the parameters

> **Key point:** The recurrent layer has 1,088 parameters and the output layer has 33. The table adds them up.

The notation is that of RNN forward propagation: W_i holds the weights from the input to the layer's nodes, W_h the feedback weights from the nodes' previous hidden state back to the nodes, b_h their biases, and W_o, b_o the weights and bias of the output node.

Part	Count
W_i: 1 input feature to 32 nodes	$1 \times 32 = 32$
W_h: feedback, 32 nodes to 32 nodes	$32 \times 32 = 1024$
b_h	32
SimpleRNN total	**1,088**
W_o and b_o	$32 + 1 = 33$

`model.summary()` prints the same numbers.

5.3 `return_sequences`

> **Key point:** `return_sequences=False` (the default) returns only the last hidden state. `return_sequences=True` returns the hidden state of every time step.

The recurrent layer computes a hidden state at every time step (Figure 4). The argument `return_sequences` (G-136) chooses which of them leave the layer. With `return_sequences=False`, only the last one leaves the layer; the others stay inside and only feed the next time step. **Sentiment analysis** (G-1769) needs one answer per review, after the last word, so `False` is right here.

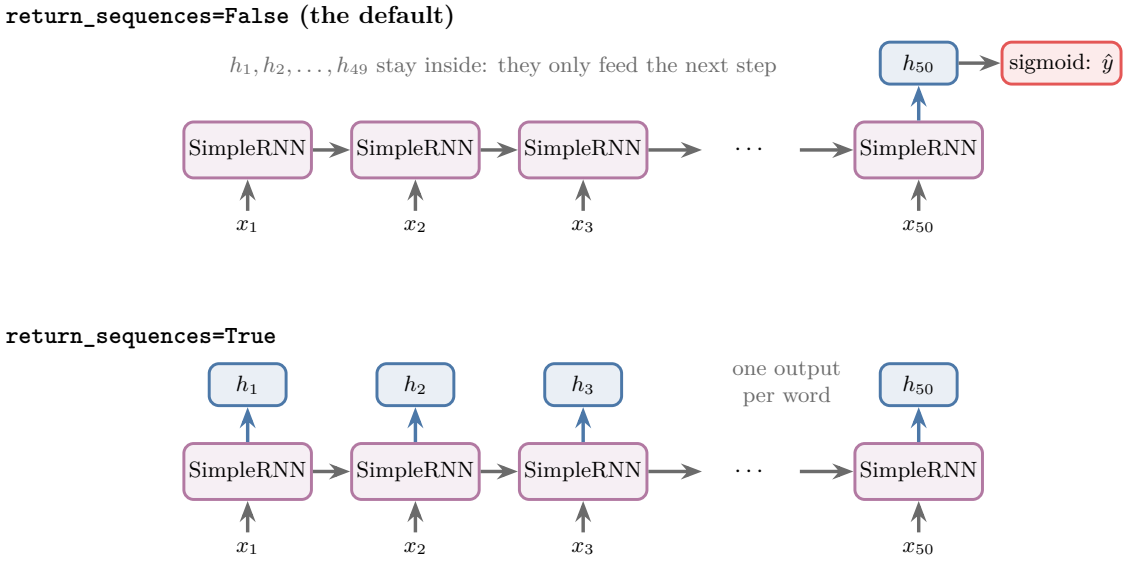


Figure 4: The SimpleRNN of section 5 unrolled over a review's 50 integers. Top: with `return_sequences=False` only h_{50} goes on to the sigmoid output. Bottom: with `True` every hidden state leaves the layer, one per word

Some tasks need an output at every word: **named entity recognition** (G-1300) labels each word (for example, is it a person's name?), and **machine translation** (G-1141) produces a sentence. Those use `return_sequences=True`. The types of RNN covers these cases.

5.4 Training

> **Key point:** Binary cross-entropy, Adam, 5 epochs, the test reviews as validation data. Raw integers give a test accuracy near a coin toss.

We compile with **binary cross-entropy** (G-303; the loss for two classes) loss and the **Adam** (G-169; the rule that updates the weights) optimizer, train for 5 **epochs** (G-696) and pass the test reviews as `validation_data`. Averaged over 3 runs, the model reaches 0.50 training accuracy and 0.50 test accuracy after 5 epochs (grey lines in Figure 1). With two balanced classes, guessing gives 0.50.

Only 50 words per review and only 5 epochs make the task harder. But the embedding model of section 7 gets the same 50 words and the same 5 epochs and reaches 0.80. The main difference between the two models is how a word enters the RNN (the embedding model also

keeps only the 10,000 most frequent words), so the representation of the words is what holds this model back.

Why raw integers are a poor input. The integer of a word is only its rank in a frequency list. In the IMDB data "great" is 87, "excellent" is 321 and "awful" is 373 (Notebook). Read as quantities, these numbers say that "excellent" is almost the same as "awful" and far from "great". Two words with the same meaning get unrelated numbers, so what the network learns about "great" does not carry over to "excellent". Section 6 replaces the single arbitrary number by a few learned ones.

6. Word embeddings

> **Key point:** An embedding turns each word into a short vector of real numbers, most of them non-zero, learned so that words used in similar ways get similar vectors.

6.1 Sparse and dense representations

> **Key point: Padding** (G-1436) and one-hot vectors are mostly zeros (sparse). An embedding is short and full of useful numbers (dense).

Two problems come with the earlier representations.

- **Sparse.** Take a 20-word review padded to the longest review of 2,000 words: 1,980 of its 2,000 numbers are padding zeros. A one-hot vector over a 10,000-word **vocabulary** (G-2092) is 9,999 zeros and one 1. A representation where most values are 0 is **sparse** (G-1845).
- **No meaning.** Integers and one-hot vectors say nothing about meaning. In one-hot space every pair of words is the same distance apart, so "good" is as far from "great" as from "awful" (Goodfellow §12.4.2).

Two one-hot vectors differ in exactly two places, each by 1, so the distance between any two words is

$$\sqrt{1^2 + 1^2} = \sqrt{2}$$

A **word embedding** (G-2127) represents each word as a real-valued vector of a chosen, small size, such as 2 or 32 numbers, most of them non-zero: a **dense** representation (G-585). The vectors are learned so that words that appear in similar contexts are close to each other, which often puts words with similar meanings next to each other (Goodfellow §12.4.2).

6.2 Learning the embedding with the model

> **Key point:** Keras' `Embedding` layer learns the vectors during training, by **backpropagation** (G-247), together with the RNN.

Word2Vec (Mikolov et al. 2013) and **GloVe** (G-851; Pennington et al. 2014) are well-known embedding methods trained on large text collections: Word2Vec learned its vectors from a text of 1.6 billion words, and GloVe was trained on texts of 1 to 42 billion tokens. In deep learning we can also learn an embedding as part of our own model: Keras' `Embedding` layer turns each integer into a dense vector of fixed size (Keras documentation, `Embedding`). Its input must be integer encoded.

`Embedding` takes two main arguments:

- `input_dim`: the **vocabulary** (G-2092) size;
- `output_dim`: the size of each word's vector, a **hyperparameter** (G-910) (2, 32, 200, ...) to tune.

> **Extra:** Older code also passes `input_length`, the number of words per sequence. Keras 3 removed that argument; the sequence length comes from the input shape (`keras.Input(shape=(50,))`) instead (Keras documentation, `Embedding`).

6.3 What the layer computes

> **Key point:** The layer holds a weight matrix E with one row per word. A word's vector is its row of E: a one-hot vector times E, done as a lookup.

1. **In words:** the layer is like a dense layer whose input is the word's one-hot vector, with no bias and no activation. Multiplying a one-hot vector by a matrix picks one row, so the layer simply looks up row k for word k.
2. **Formula:** for a **vocabulary** (G-2092) of V words and vectors of size d, E is $V \times d$, and
$$\text{embedding}(k) = \text{onehot}(k)\, E = E_{k,:}$$
Here $E_{k,:}$ means row k of E: the entry k in each column, the whole row of numbers.
3. **Example:** the slogan document has $V = 19$ entries. With $d = 2$, E is 19×2: 38 weights, which `model.summary()` confirms. The first word of the first slogan, "go", has index 16, and row 16 of E is its 2-number vector: $[-0.039, -0.006]$ before training.

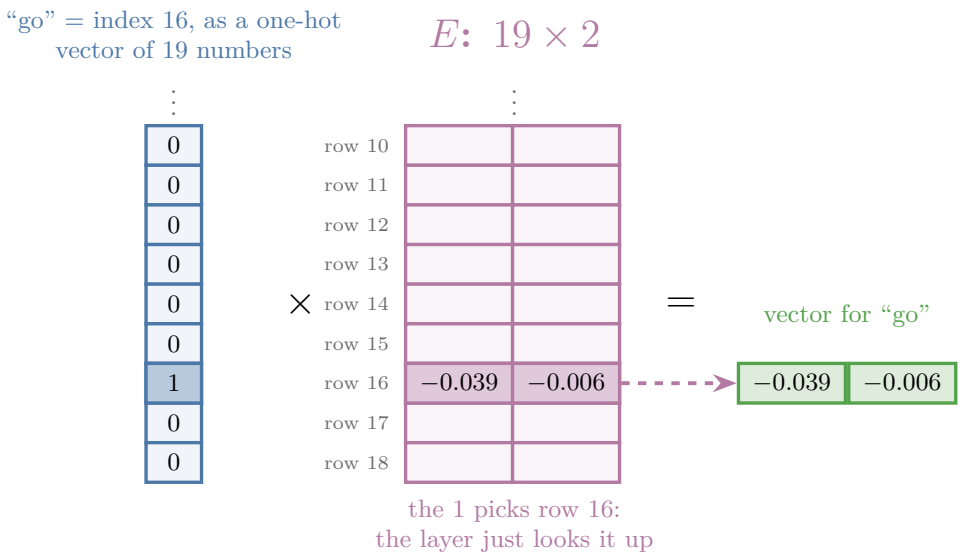


Figure 5: The embedding layer as a lookup: the one-hot vector of "go" (index 16, drawn standing up to save space; in the product it is a row of 19 numbers) times E keeps row 16 of E, which becomes the word's dense vector

E starts random and is trained with the rest of the network by **backpropagation** (G-247). Figure 5 shows the lookup. The Notebook checks that the one-hot product and the layer's output are the same numbers.

Applied to a padded sequence of 5 integers, the layer returns 5 rows of 2 numbers, one per word (the **padding** (G-1436) index 0 also gets a row). For the 10 slogans, padded to 5 words, the output has shape $(10, 5, 2)$: 100 numbers.

7. Approach 2: an embedding into the same SimpleRNN

> **Key point:** Keep the 10,000 most frequent words, embed each in 2 numbers, and feed the 50 vectors into the SimpleRNN. Test accuracy rises to about 0.80.

7.1 The model

> **Key point:** Embedding (20,000 parameters), SimpleRNN (1,120), Dense (33).

`load_data(num_words=10000)` keeps the 10,000 most frequent words; every rarer word becomes the out-of-vocabulary integer 2.

Python: The embedding model.

```
model = keras.Sequential([
    keras.Input(shape=(50,)),              # 50 integers per review
    keras.layers.Embedding(10000, 2),      # each word -> 2 numbers
```

```
    keras.layers.SimpleRNN(32),
    keras.layers.Dense(1, activation="sigmoid")])
```

Now each time step carries 2 numbers instead of 1, so the RNN's input is (50 time steps, 2 input features).

Layer	Parameters
Embedding: E	$10000 \times 2 = 20000$
SimpleRNN: W_i, W_h, b_h	$2 \times 32 + 32 \times 32 + 32 = 1120$
Dense: W_o, b_o	$32 + 1 = 33$

7.2 Results

Key point: Same RNN, same 50 words, same 5 **epochs** (G-696): the embedding lifts test accuracy from 0.50 to 0.80. Training accuracy climbs higher still, a sign of overfitting.

After 5 epochs, mean of 3 runs	Integer encoding	Embedding
Training accuracy	0.50	0.90
Test accuracy	0.50	0.80

The green lines of Figure 1 show the embedding model. Test accuracy levels off at about 0.80 from epoch 2 while training accuracy keeps rising to 0.90: the model is starting to memorise the training reviews, which is **overfitting** (G-1429). The regularization and the early stopping cover the remedies.

7.3 What the embedding learned

Key point: After training, positive and negative words sit in different parts of the 2-number space.

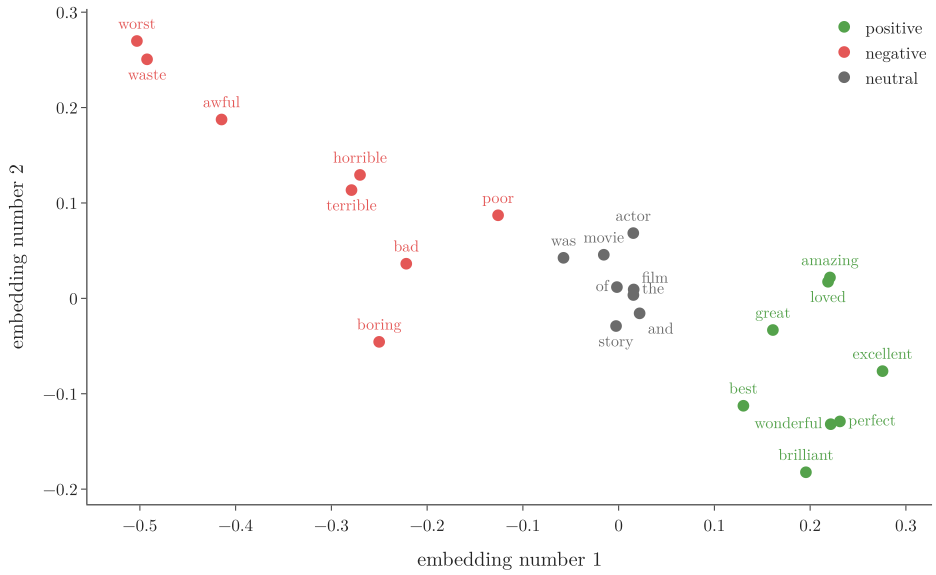


Figure 6: The learned 2-number vectors of a few IMDB words after training. Positive words (green) and negative words (red) end up on opposite sides; frequent neutral words (grey) sit between them

Figure 6 plots the learned vectors of 8 positive, 8 negative and 8 neutral words: the two numbers of each vector give the horizontal and vertical position of its word. Nobody told the network which words are positive. Goodfellow §12.4.2 explains the pattern: words that share features learned by the model end up close together. The only label this model learns from is the sentiment, and Figure 6 shows its embedding sorting the words by sentiment along one direction.

How the vectors get there. Figure 7 shows the same 24 words before training and after each **epoch** (G-696). Watch the green and red points: they start mixed in one small cloud and move apart, mostly during the first epoch.

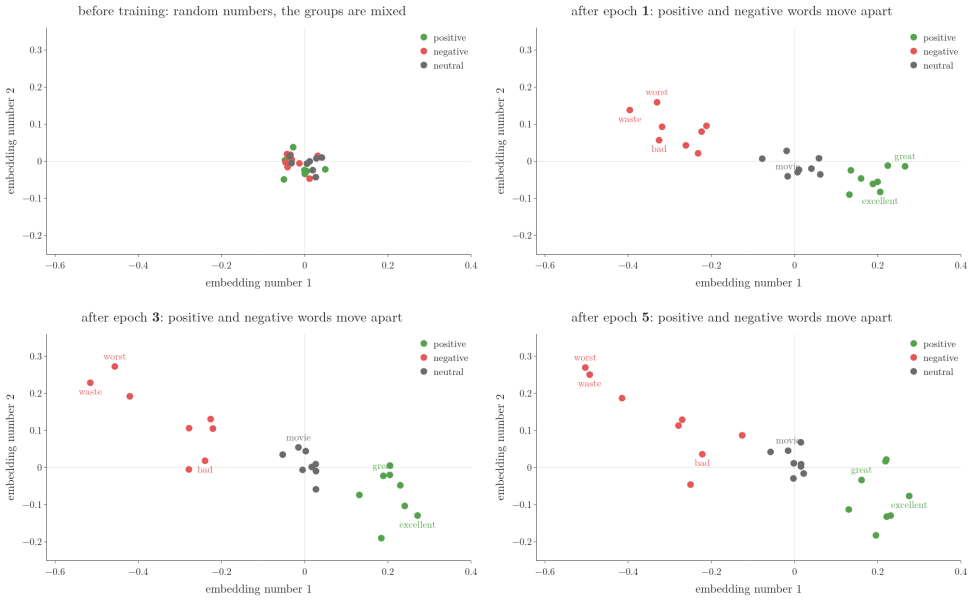


Figure 7: The embedding of the model of Figure 6 during training (seed 0): the 24 words before training and after each of the 5 epochs. The vectors start as small random numbers; training moves positive words (green) one way and negative words (red) the other, while neutral words (grey) stay near the middle (idea after StatQuest, "Word Embedding and Word2Vec, Clearly Explained!!!")

1. **Before training,** E holds small random numbers, so the positions say nothing: the average first number is -0.01 for the positive words and -0.02 for the negative words (Notebook).
2. **At every training step, backpropagation** (G-247) changes the rows of E for the words of the reviews in the batch, each in the direction that lowers the loss. The rows of E are weights like any others.
3. **Words used in the same kind of review get the same kind of push.** "Great" and "excellent" appear mostly in positive reviews, so their rows are moved the same way; "worst" and "awful" are moved the opposite way. After one epoch the average first number is 0.19 for the positive words and -0.29 for the negative words.

Nobody placed similar words together: they end up together because they help the prediction in the same way.

The trained model can also be watched while it reads. We take the first test review of 20 to 30 words, with no rare word, that the model classifies correctly (Notebook, last section), and record the hidden state after every word. Applying the output layer to each hidden state gives the prediction the model would make if the review stopped there.

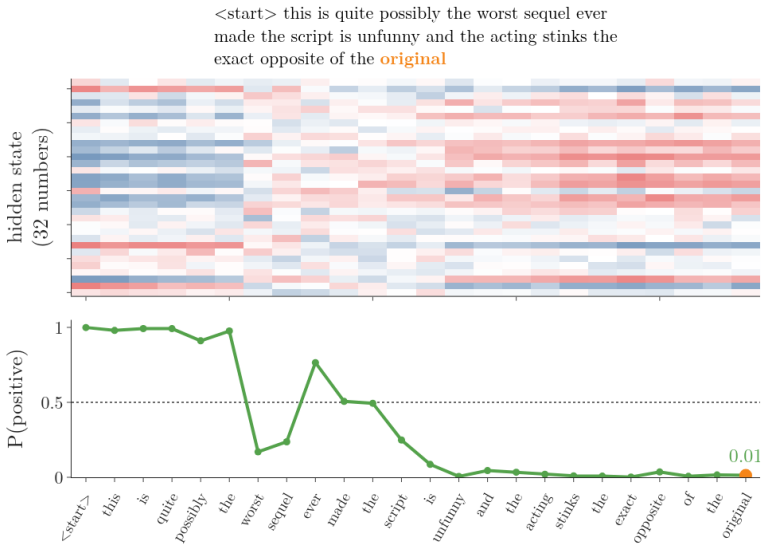


Figure 8: The trained embedding model reads a negative test review word by word. Top: the 32 numbers of the hidden state after each word (blue positive, red negative). Bottom: the prediction if the review stopped at that word. The prediction falls from 0.98 to 0.17 at "worst", climbs back to 0.77 at "ever", then falls for good and ends at 0.013

In Figure 8, watch the column for "worst": the hidden state changes colour there. The next words pull the prediction part of the way back, and "script is unfunny" settles it on the negative side.

7.4 Learned or pre-trained embeddings

> **Key point:** We can learn the embedding with the model, as here, or load pre-trained vectors such as Word2Vec or GloVe.

A pre-trained embedding, like a **pretrained model** (G-1558), was learned on a large general text collection. An embedding learned with the model is fitted to our own data and task. Which works better depends on the data (Goodfellow §15.1.1):

- Pre-training helps most when the labelled observations are few, and when the text used for pre-training is very large.
- Words are a case where pre-training is especially useful, because one-hot words carry no similarity information (section 6.1). In natural language processing, the vectors are often learned once on a huge text of billions of words, then used as they are or fine-tuned for the task.

8. Summary

	Integer encoding	Embedding
A word becomes	1 number, its index	a learned vector of d numbers
Representation	arbitrary numbers, sparse after padding	dense, similar words close
RNN input shape	(50, 1)	(50, d)
Extra parameters	none	vocabulary size $\times$ d
IMDB test accuracy, 5 epochs	0.50	0.80

- Text must become numbers, because an RNN only reads numbers: build a **vocabulary** (G-2092), replace words by integers, pad to one length so the sequences fit in one batch.
- `TextVectorization` tokenizes and integer encodes; unknown words map to `[UNK]`, so a word seen only at prediction time still gets an index; `pad_sequences` pads and cuts.
- IMDB comes integer encoded; `pad_sequences(maxlen=50)` keeps the last 50 words of each review, because it cuts from the start by default (`truncating="pre"`).
- `SimpleRNN(32)` on 1 input feature has 1,088 parameters; `return_sequences=False` returns only the last hidden state, because sentiment needs one answer per review, after the last word.
- An `Embedding` layer is a lookup table E, trained with the model; it turned a coin toss into about 0.80 accuracy, because a word's integer is only its frequency rank, while the learned vectors put words that help the prediction in the same way close together.

9. Sources

Built from

- CampusX, "RNN Sentiment Analysis | RNN Code Example in Keras | CampusX", YouTube, https://www.youtube.com/watch?v=JgnbwKnHMZQ
- StatQuest with Josh Starmer, "Word Embedding and Word2Vec, Clearly Explained!!!", YouTube, https://www.youtube.com/watch?v=viZrOnJclY0. 02:00–10:00 (arbitrary numbers for words of the same meaning; the embedding as weights learned by back-propagation; words plotted by their weights before and after training).

Other references

- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §12.4.2 (neural language models, word embeddings). deeplearningbook.org/contents/applications.html.
- Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. MIT Press. §15.1.1 (when unsupervised pretraining works). deeplearningbook.org/contents/representation.html.
- Mikolov, T., Chen, K., Corrado, G. and Dean, J. (2013). Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781.
- Pennington, J., Socher, R. and Manning, C. D. (2014). GloVe: Global Vectors for Word Representation. EMNLP 2014, 1532-1543.

- Keras API documentation: `TextVectorization`, `pad_sequences`, `Embedding`, `SimpleRNN` layers, and the IMDB dataset (`keras.datasets.imdb.load_data`), keras.io/api/. Checked against the installed Keras 3.15 source.
- Maas, A. L. et al. (2011). Learning Word Vectors for Sentiment Analysis. ACL 2011. (The IMDB dataset.)

10. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Padding (G-1436)	Adding zeros to sequences so that all have the same length.
Integer encoding (G-956)	Replacing each word by its index (a whole number) in the vocabulary, so a network such as an RNN can read text as numbers.
Out-of-vocabulary (OOV) token (G-1416)	A placeholder token, `[UNK]`, that stands for every word not in the vocabulary, so a word never seen in training still gets an index.
IMDB dataset (G-923)	50,000 film reviews labelled positive or negative, a standard sentiment-analysis dataset.
`return_sequences` (G-136)	Argument of a Keras recurrent layer: `False` returns only the last hidden state, `True` returns the hidden state at every time step; `True` is needed on every stacked layer but the last.
Sparse representation (G-1845)	A representation where most values are 0, such as a one-hot vector over a 10,000-word vocabulary (9,999 zeros and one 1); it wastes space compared with a dense one.
Word embedding (G-2127)	A learned real-valued vector for each word; words used in similar ways get nearby vectors.
Dense representation (G-585)	A short vector of real numbers, most of them non-zero, such as a word embedding; unlike a long, mostly-zero one-hot vector, every number carries information.
`Embedding` layer (G-77)	The Keras layer that holds a table with one learned vector per word (the embedding matrix E) and looks up one row per word.
Embedding (G-677)	A learned vector representing a user, item or word, compared with others by dot products.

Term	Meaning
Vocabulary (V) (G-2092)	The list of distinct words (tokens) in a set of texts; in a translation model, those of the target language, with one output node per word.
Sentiment analysis (G-1769)	Deciding whether a text expresses a positive or negative opinion.
Epoch (G-696)	One full update of the parameters using the whole training set.
One-hot encoding (G-1379)	Replacing a nominal column by one 0/1 column per category, with a single 1 in each row; also used to represent words.
Observation (G-1374)	One record of a dataset: one row of the data table.
Target (G-1949)	The output column we predict, such as the class; also called the label.
Tokenisation (tokenization) (G-1983)	Splitting a text into tokens (words or word pieces), the units a language model reads and writes, before they are turned into vectors.
Named entity recognition (NER) (G-1300)	Marking which words of a sentence are entities, specific things a program must act on, such as "7 pm" and "airport"; chatbots use it to pick out times and places.
Machine translation (G-1141)	Translating a sentence from one language into another.
Binary cross entropy (log loss) (G-303)	The loss function of logistic regression for two classes: per row $-y \log a - (1-y)\log(1-a)$, averaged over the rows (the average cross entropy).
Adam (G-169)	An optimizer, the rule that updates a network's weights to reduce the loss: a variant of gradient descent that keeps running averages of past gradients and of their squares, giving each weight its own step size; fairly robust to its settings, so a common default.
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.

Term	Meaning
GloVe (G-851)	Word vectors learned from how often words appear near each other: training makes the dot product of two words' vectors match the log of their co-occurrence probability (Pennington et al. 2014).
Hyperparameter (G-910)	A setting of an algorithm chosen before training, such as a tree's `max_depth`.
Overfitting (G-1429)	Learning the training data too closely, noise included; fails on new data.
Pretrained model (G-1558)	A network already trained by someone else on a large dataset, reused for our own predictions.